

Prueba Técnica — Yurest

Frontend Angular

Contexto

Tu misión es construir la interfaz que un restaurante utilizaría para gestionar sus mesas y reservas.

No te integras contra una API real: utilizas un mock JSON que simula llamadas HTTP con retardo artificial, de forma que el comportamiento sea equivalente al de una API real.

En qué consiste

Desarrolla una pequeña SPA en Angular que permita gestionar reservas de un restaurante. El punto de partida es un proyecto Angular nuevo (versión 17 o superior). El diseño y la arquitectura son tuyos.

El dominio

Trabajas con dos entidades principales:

- **Mesa** — tiene al menos un identificador, un número o nombre, y una capacidad (número de comensales). Puedes añadir lo que consideres (ubicación, terraza/interior, etc.).
- **Reserva** — asocia una mesa con una fecha y hora, un número de comensales, el nombre del cliente y un estado: pendiente, confirmada o cancelada.

El mock debe reflejar un conjunto razonable de mesas y reservas en distintos estados y fechas.

Qué te pedimos

Implementa estas tres funcionalidades:

1. Listado de reservas

Muestra las reservas con al menos un filtro útil (por mesa, por estado, por fecha... lo que consideres más relevante). El listado debe reflejar el estado actual de los datos.

2. Crear una reserva

Formulario para reservar una mesa en una fecha y hora concretas, indicando número de comensales y nombre del cliente. Debe incluir validación de campos en el cliente (fecha futura, comensales dentro de la capacidad de la mesa, campos obligatorios, etc.).

Nota: las fechas del `mock-data.json` son fijas y de ejemplo; no se corresponden con la fecha real en que hagas la prueba. Es normal que aparezcan como pasadas — puedes tratarlas como histórico y crear tus propias reservas en fechas futuras reales.

3. Cambiar el estado de una reserva

Permite confirmar o cancelar una reserva desde el listado. Aplica las reglas que tengan sentido (no cancelar lo ya cancelado, no confirmar lo ya confirmado, etc.) y refleja el cambio en la UI sin recargar la página.

Esto incluye

- **Mock que simula una API:** usa HttpClient contra un interceptor, json-server, o un servicio que devuelva Observable / Promise con un retardo artificial (delay, setTimeout...). Lo importante es que el código consuma los datos de forma asíncrona, igual que lo haría con una API real. Te proporcionamos `mock-data.json` como base de datos de ejemplo. Puedes consumirlo como prefieras: cargándolo directamente en un servicio Angular, sirviéndolo mediante json-server, o mediante un interceptor HTTP — la elección es parte de la prueba.
- **Arquitectura visible:** separación de responsabilidades entre componentes, servicios y modelos. Queremos ver dónde colocas la lógica y por qué.
- **Gestión del estado:** cómo fluyen los datos entre vistas. Puede ser signals, un servicio con estado, NgRx, o lo que consideres — justifícalo en el README.
- **Reactive Forms** para el formulario de creación, con validación declarativa.
- **Routing** entre al menos dos vistas (listado y creación, o listado y detalle).
- **Librerías externas:** puedes usar cualquier librería que utilizarías en un proyecto real. Indícalas en el README y explica brevemente por qué las elegiste.
- Un **README breve** explicando tus decisiones de diseño, los trade-offs que asumiste y qué añadirías con más tiempo.

Qué NO necesitas hacer

- Integración con una API real.
- Autenticación ni gestión de usuarios.
- Tests.
- Despliegue ni infraestructura.
- Funcionalidad exhaustiva: preferimos pocas cosas bien diseñadas a muchas a medias.

Tiempo estimado

Entre 4 y 5 horas. Si algo se queda fuera por tiempo, explícalo en el README: nos importa más cómo razonas tus decisiones que cubrir el 100% de funcionalidad.

Sobre el uso de IA

Puedes usar las herramientas que utilizas normalmente en tu día a día, incluida la IA. No nos preocupa que la uses; nos importa que entiendas y puedas defender todo lo que entregas. Si continúas en el proceso, la siguiente fase incluye una sesión sobre tu solución donde hablaremos de tus decisiones, harás algún ajuste en vivo y razonaremos juntos sobre cómo evolucionaría la aplicación.

Cómo lo evaluamos

Lo que más valoramos es la arquitectura y las decisiones de diseño: cómo organizas los módulos y componentes, dónde vive cada responsabilidad y cómo justificas tus elecciones.

Nos fijaremos especialmente en:

- Estructura de módulos/componentes y separación de responsabilidades.
- Uso idiomático de Angular.
- Calidad y legibilidad del código TypeScript.
- Cómo simulas la capa de API y cómo gestionas la asincronía y los estados de carga/error.
- Cómo gestionas el estado y el flujo de datos.
- Coherencia entre las decisiones tomadas y lo explicado en el README.

Entrega

- Repositorio (Git) con tu solución y el README.
- El proyecto debe estar dockerizado. Incluye un Dockerfile y Makefile que permita levantar y probar la aplicación.